Simón, con Raspberry Pi

Ventana gráfica:

Para disponer los elementos de la aplicación en la ventana gráfica se usará una rejilla. La distribución de la rejilla quedará de la siguiente manera:

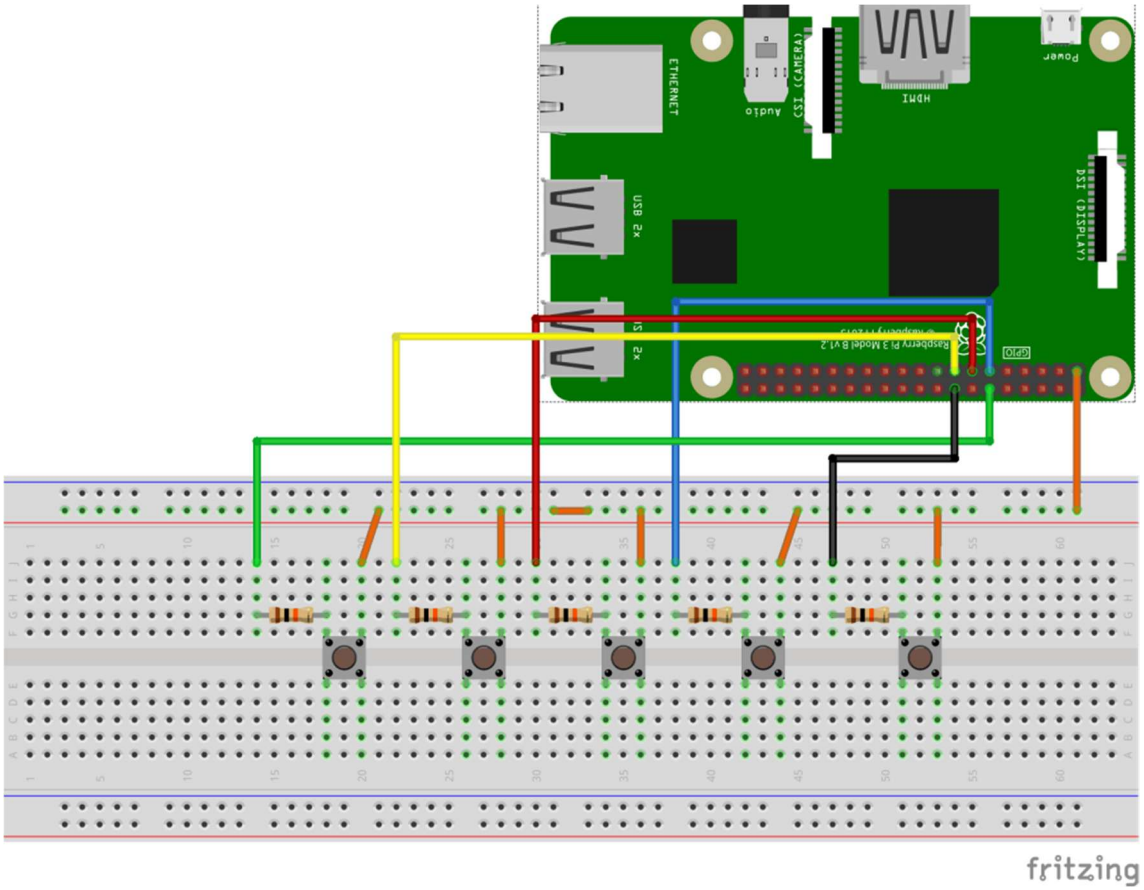


Un primera fila (en rojo), contendrá cuatro columnas para mostrar las imágenes que simulan los leds del juego.

Bajo las imágenes se encuentran dos filas más (en azul), ocupando todo el ancho de la ventana, para mostrar mensajes al jugador.

En la parte inferior, están las dos últimas filas (en verde), las cuales contienen dos columnas cada una, para mostrar la puntuación de la partida actual y la máxima alcanzada.

Esquema (Fritzing):



Conexiones:

Cada pulsador está conectado a 3.3v y a un pin del GPIO. Entre el pulsador y el pin del GPIO se introduce una resistencia de 10kohms para reducir la corriente que se envía al pin.

Botón	**Pin**
Verde	12
Amarillo	15
Rojo	13
Azul	11
Negro	16

Librerías:

Se necesitarán tres librerías externas para la programación del juego.

- **RPi.GPIO**: para el control del GPIO de la Raspberry
- **Tkinter**: para crear interfaces gráficas
- **PIL**: para poder trabajar con ficheros de imagen

RPi.GPIO y Tkinter suelen estar ya instaladas en las distribuciones de Raspbian. En caso contrario contrario, se puede encontrar información de las librerías y su instalación en los siguientes enlaces:

- **Información de RPi.GPIO**: https://pypi.org/project/RPi.GPIO/
- **Instalación de RPi.GPIO**: https://www.raspberrypi-spy.co.uk/2012/05/install-rpi-gpio-python-library/
- **Información de Tkinter**: https://wiki.python.org/moin/TkInter
- **Instalación de Tkinter**: https://python-para-impacientes.blogspot.com/2015/12/tkinter-interfaces-graficas-en-python-i.html

PIL se ha de instalar. Usando apt-get, según la versión de Python, se tendrá que ejecutar:

Para versiones anteriores a la 3 de Python:

```
sudo apt-get install python-imaging python-imaging-tk
```

Para la versión 3 de Python:

```
sudo apt-get install python3-pil python3-pil.imagetk
```

Código:

Primero, importamos los módulos que va a usar la aplicación:

Importaciones

```
from PIL import Image, ImageTk
from tkinter import Tk, ttk, Label
import sys
from time import sleep
import RPi.GPIO as GPIO
from random import randint, seed
```

- Image, ImageTk: se usarán para la gestión de las imágenes de la aplicación
- Tk, ttk, Label: se usarán para crear la interfaz gráfica y las etiquetas que mostrarán información en la aplicación
- sys: se usará para terminar la ejecución de la aplicación
- sleep: se usará para generar pausas durante la ejecución de la aplicación
- RPi.GPIO: se usará para manejar el GPIO
- randint, seed: se usarán para generar números aleoatorios

Se declara la clase que contendrá el código de la aplicación y sus propiedades:

Simon

```
class Simon():
# contenedor para la ventana de tkinter
raiz = None

# botones
# botón de inicio (negro)
botonInicio = None

# botones de colores:
botonesColores = []

# colores del juego
colores = ('verde', 'amarillo', 'rojo', 'azul')

# listas para los objetos de imagen de los colores
coloresOn = [None, None, None, None]
coloresOff = [None, None, None, None]

# etiquetas de tkinter para mostrar las imagenes de les colores
cajasColores = [None, None, None, None]

# mensajes que muestra la aplicación al jugador
mensajes = {
          "bienvenida.1": "Bienvenido a Simon",
          "bienvenida.2": "Pulsa el botón negro para empezar",
          "instrucciones.1": "Repite la secuencia de colores",
          "instrucciones.2": "¡Empezamos!",
          "secuencia.1": "Secuencia {0}",
          "secuencia.2": "Repite la secuencia",
          "puntos": "Puntos",
          "record": "Record",
          "juego.terminado": "Juego terminado",
          "intento.ok": "¡Bien! Siguiente secuencia",
          "intento.ko": "¡Oh! ¡Has fallado! "
}

# labels de tkinter donde se muetran los mensajes la jugador
cajasMensajes = [None, None]

# para controlar si la partida esta iniciada
partidaIniciada = False

# para controlar si esta disponible la funcionalidad de los botones de colores
botonesBloqueados = True

# contadores para puntos y records
puntos = 0
record = 0

# etiquetas de tkinter para las puntuaciones (títulos y valores)
cajaTituloPuntos = None
cajaTituloRecord = None
cajaPuntos = None
cajaRecord = None

# secuencia de colores de la partida
secuencia = []

# punto de la secuencia a comprobar con la pulsación del jugador
pasoComprobar = 0
```

on_closing

Definimos una método para ejecutar cuando el jugador cierre la aplicación. Este método estará relacionado con el manejador de protocolo WM_DELETE_WINDOW y se encargará de eliminar la interfaz gráfica, limpiar la configuración de la GPIO y salir de la aplicación.

```
##
# on_closing
# acciones a realizar al cerrar la aplicación:
# -destruye la ventana gráfica
# -limpia la configuración de GPIO
# -sale de la aplicación
##
def on_closing(self):
self.raiz.destroy()
GPIO.cleanup()
sys.exit()
```

anyadirEventosColores

Se definen los eventos que se han de ejecutar cada vez que se pulse un botón.

Para esto usaremos el método **add_event_detect**, de **GPIO**. Este método admite cuatro parámetros:

- el pin del botón que se va a asociar al evento.
- el modo de detección: define cómo se va a detectar que se ha pulsado el botón. Admite tres valores: **GPIO.RISING**, para detectar cuando el botón pase de 0 a 1, **GPIO.FALLING**, para detectar cuando el botón pase de 1 a 0 y **GPIO.BOTH**, para detectar ambos cambios.
- el callback o función a llamar cuando se produzca el evento. Se indica asignando un valor a la variable **callback**.
- bouncetime o tiempo de retardo: es el tiempo que se ha de esperar para volver a atender eventos del botón asignado. Esto permite reducir los rebotes producidos al pulsar el botón. Se indica en milisegundos y se debe asignar a la variable **bouncetime**.

Se asigna un evento para el botón negro y otro para los cuatro botones de colores.

```
##
# anyadirEventosBotonesColores
# añade el evento a ejecutar cuando se pulsa uno de los botones de colores
# se dispara al cambiar su estado de 0 a 1
##
def anyadirEventosBotonesColores(self):
          for boton in self.botonesColores:
                    GPIO.add_event_detect(boton, GPIO.RISING, callback =
self.comprobarBotonColor, bouncetime = 250)
```

anyadirEventoIniciar

```
##
# anyadirEventoIniciar
# añade el evento a ejecutar cuando se pulsa el botón de inicio
# se dispara al cambiar su estado de 0 a 1
##
def anyadirEventoIniciar(self):
          GPIO.add_event_detect(self.botonInicio, GPIO.RISING, callback =
self.iniciarJuego, bouncetime = 1000)
```

iniciarImagenes

Esta vez no vamos a usar leds (como se hizo en la aplicación con Arduino), si no que los sustituiremos por imágenes. Tendremos cuatro por cada estado, encendido y apagado. Los nombres de las imágenes serán verde_on.png, amarillo_on.png, rojo_on.png y azul_on.png para imitar los leds encendidos y verde_off.png, amarillo_off.png, rojo_off.png y azul_off.png para imitar los leds encendidos.

Definimos un método para cargar las imágenes y almacenarlas en dos listas (una para cada estado). Este método recibe como parámetro el tipo de imágenes que se han de cargar, sean las que imitan encendido (‘on’) o las que imitan apagado (‘off’).

```
##
# iniciarImagenes
# carga los ficheros con las imagenes que hacen de colores y los almacena en listas
# @tipo el tipo de imagen que se va a cargar: 'on' para los colores 'encendidos', 'off'
para los colores 'apagados'
##
def iniciarImagenes(self, tipo):
          for indice, valor in enumerate(self.colores):
                    imagen = Image.open(valor + "_" + tipo + ".png")
                    if(tipo == "on"):
                              self.coloresOn[indice] = ImageTk.PhotoImage(imagen)
                    else:
                              self.coloresOff[indice] = ImageTk.PhotoImage(imagen)
```

La aplicación va a mostrar las imágenes que hacen de leds, mensajes de información al jugador y su puntuación y record. Para hacer esto necesita elementos dentro de la ventana gráfica. Vamos a definir algunos métodos que se encarguen de montar estos elementos.

montarImagenes

```
##
# montarImagenes
# monta las etiquetas para mostrar las imagenes con los colores del juego
##
def montarImagenes(self):
          for indice, valor in enumerate(self.coloresOff):
                    self.cajasColores[indice] = Label(self.raiz, image =
          self.coloresOff[indice])
          self.cajasColores[indice].grid(row = 0, column = indice, padx = 5, pady = 5)
          self.raiz.columnconfigure(indice, weight = 1)
```

Este método recorre la lista de imágenes que imitan los leds apagados y, por cada una de ellas, se genera una etiqueta mediante el método **Label** de Tkinter.

El método **Label** espera recibir como parámetros el contenedor donde se va a alojar la etiqueta y los valores para definir sus opciones (ver una lista aquí). Para las imágenes le pasamos como contenedor la propia ventana de la aplicación y la opción **image**, asignándole la imágen del led que corresponda.

Seguidamente lo añade a la rejilla de la ventana gráfica, usando el método **grid**, pasándole como argumentos la fila y la columna donde se ha de posicionar la imagen y la separación interna horizontal y vertical. Finalmente, modifica la configuración de las columnas del contenedor (en este caso es la ventana de la aplicación en sí), para que ocupe todo el ancho disponible, mediante el parámetro **weight**. Al ser cuatro botones, esta configuración hará que se reparta el ancho de la ventana por igual para las cuatro imágenes.

montarMensajes

```
##
# montarMensajes
# monta las etiquetas en la rejilla de la ventana para mostrar los mensaje de información
al jugador
##
def montarMensajes(self):
          for indice, valor in enumerate(self.cajasMensajes):
                    self.cajasMensajes[indice] = Label(self.raiz, text = "");
                    self.cajasMensajes[indice].grid(row = (indice + 1), column = 0,
columnspan = 4, pady = 10)
```

Este método sitúa las etiquetas para mostrar los mensajes al jugador justo debajo de las imágenes que hacen de leds. Para hacer que ocupe la misma anchura que ocupan las cuatro etiquetas de las imágenes, al llamar al método **grid** para cada etiqueta, se le pasa la opción columnspan, indicándole el número de columnas que ha de ocupar. En este enlace se pueden consultar las diferentes opciones que puede recibir el método **grid**.

montarPuntuaciones

```
##
# montarPuntuaciones
# monta las etiquetas en la rejilla de la ventana para mostrar las puntuaciones del juego
##
def montarPuntuaciones(self):
          self.cajaTituloPuntos = Label(self.raiz, text = self.mensajes["puntos"])
          self.cajaTituloPuntos.grid(row = 3, column = 0, columnspan = 2, pady = 5)
          self.cajaTituloRecord = Label(self.raiz, text = self.mensajes["record"])
          self.cajaTituloRecord.grid(row=3, column=2, columnspan=2, pady = 5)
          self.cajaPuntos = Label(self.raiz, text = self.puntos)
          self.cajaPuntos.grid(row = 4, column = 0, columnspan = 2, pady = 5)
          self.cajaRecord = Label(self.raiz, text = self.record)
          self.cajaRecord.grid(row=4, column=2, columnspan=2, pady = 5)
```

Definidos los métodos que montan los componentes de la rejilla, vamos a definir un método que permita modificar el texto de las etiquetas que se han añadido para mostrar mensajes al jugador.

mostrarMensaje

```
##
# mostrarMensaje
# muestra un mensaje en alguna de las dos etiquetas disponibles
# @index: indice de la etiqueta donde mostrar el mensaje
# @mensaje: indice de la lista de mensajes que se ha de mostrar (indicar None para borrar
el mensaje actual)
# @parametros (opcional): lista de parametros que se han de incluir en el mensaje,
mediante la función 'format'
##
def mostrarMensaje(self, index, mensaje, parametros = None):
          texto = ""
          if(mensaje != None):
                    texto = self.mensajes[mensaje]
                    if(parametros != None):
                              texto = texto.format(*parametros)
          self.cajasMensajes[index].configure(text = texto)
```

Este método recibe tres parámetros:

- index: índice, dentro de la lista de etiquetas para mostrar mensajes, que se va a modificar.
- mensaje: índice dentro de la propiedad mensajes de la clase que contiene el mensaje a mostrar al jugador.
- parámetros (opcional): una matriz con valores, los cuales se añadirán al mensaje a mostrar.

Mediante el método configure del objeto Label, se modifica la opción **text** para que se muestre el nuevo mensaje.

Constructor (__init__)

```
##
# __init__
##
def __init__(self, pinInicio, pinVerde, pinAmarillo, pinRojo, pinAzul):
        # se asignan los valores para los pins recibidos al instanciar la clase
        self.botonInicio = pinInicio
        self.botonesColores = [pinVerde, pinAmarillo, pinRojo, pinAzul]
        # inicialización del GPIO
        GPIO.setmode(GPIO.BOARD)
        # configuración de los pines como entrada de datos
        GPIO.setup(self.botonInicio, GPIO.IN, pull_up_down = GPIO.PUD_DOWN)
        GPIO.setup(self.botonesColores, GPIO.IN, pull_up_down = GPIO.PUD_DOWN)
        # se inicia un objeto de tkinter y se define su geometría y titulo
        self.raiz = Tk()
        self.raiz.geometry('300x200')
        self.raiz.title('Simón')
        # evento al cerrar la ventana
        self.raiz.protocol("WM_DELETE_WINDOW", self.on_closing)
        # se inician las imagenes (cargarlas desde disco)
        self.iniciarImagenes("on")
        self.iniciarImagenes("off")
        # se montan las etiquetas para las imagenes, mensajes y puntuaciones
        self.montarImagenes()
        self.montarMensajes()
        self.montarPuntuaciones()
        # se muetran los mensajes de bienvenida al juego
        self.mostrarMensaje(0, "bienvenida.1")
        self.mostrarMensaje(1, "bienvenida.2")
        # se añaden los eventos para las pulsaciones de los botones
        self.anyadirEventoIniciar()
        self.anyadirEventosBotonesColores()
        # inicio de la aplicación
        self.raiz.mainloop()
```

El método constructor de la clase Simon recibe cinco parámetros, los cuales son los cinco pines a los que están conectados los botones negro, verde, amarillo, rojo y azul.

Se almacenan los valores recibidos en sus variables y se inician el GPIO, asignándole los eventos a los pins recibidos como parámetros.

Se crea el contenedor gráfico y se le añaden las imágenes y mensajes de inicio.

Finalmente, se ejecuta el método **mainloop**, así, la aplicación, estará pendiente a los eventos de la ventana.

iniciarJuego

```
##
# iniciarJuego
# evento que se lanza al pulsar el botón negro
# inicia una nueva partida
# @pin: el número de pin al que esta conectado el botón negro
##
def iniciarJuego(self, pin):
          # se comprueba que la partida no este iniciada
          if(self.partidaIniciada == False):
                    # se marca la partida como iniciada
                    # y se muetran las instrucciones
                    self.partidaIniciada = True;
                    self.mostrarMensaje(0, "instrucciones.1")
                    self.mostrarMensaje(1, None)
                    sleep(0.5)
                    self.mostrarMensaje(1, "instrucciones.2")
                    sleep(0.5)
                    # se reinician la secuencia y la puntuación
                    # se lanza la primera secuencia
                    self.secuencia.clear();
                    self.puntos = 0
                    self.actualizarPuntuaciones()
                    self.lanzarSecuencia()
```

Este método se ejecuta cada vez que se pulsa el botón negro. Como parámetro, recibe el pin del botón que se ha pulsado. Este parámetro, al ser un método asociado a un evento de botón del GPIO, se obligatorio incluirlo.

Se comprueba que no se haya iniciado ninguna partida, mediante una propiedad de la clase, y, en caso que no se haya iniciado ninguna, se muestran los mensajes con las instrucciones del juego, se inician la secuencia de luces y los puntos, y, se lanza la primera secuencia del juego.

lanzarSecuencia

```
##
# lanzarSecuencia
# muestra la secuencia de colores actual
##
def lanzarSecuencia(self):
          # se obtiene el número de secuencia y se muestra como mensaje
          numeroSecuencia = len(self.secuencia) + 1
          self.mostrarMensaje(0, "secuencia.1", [numeroSecuencia])
          self.mostrarMensaje(1, None)
          sleep(0.5)
          #se elige el color a mostrar y se coloca en la última posición de la secuencia
          seed()
          color = randint(0, 3)
          self.secuencia.append(color)
          # se recorre la secuencia mostrando el color almacenado en cada posición
          for item in self.secuencia:
                    self.mostrarColor(item, True)
                    # se muestra el mensaje indicando repetir la secuencia
                    # se reinicia el control de paso de secuencia a comprobar
                    # y se desbloquean los botones de colores
                    self.mostrarMensaje(1, "secuencia.2")
                    self.pasoComprobar = 0
                    self.botonesBloqueados = False
```

Este método calcula el número de pasos que tiene actualmente la secuencia, para sumarle 1 e informar al jugador. Seguidamente elige un valor aleatorio entre 0 y 3 (índices de la lista de colores) y se añade a la secuencia, para mostrarla, mediante el método **mostrarColor** de la clase.

Una vez mostrada la secuencia, reinicia la variable que controla el paso de la secuencia que se ha de comprobar al pulsar un botón y habilita la respuesta de los botones de colores al ser pulsados.

mostrarColor

```
##
# mostrarColor
# muestra la versión 'iluminada' de un color y, tras una pausa,
# lo devuelve a su versión 'apagada'
# @color: el índice en la lista de colores que se va a mostrar
# @segundaPausa: si se ha de hacer una pausa o no después de volver a poner el color a
'apagado'
##
def mostrarColor(self, color, segundaPausa):
          self.cajasColores[color].configure(image = self.coloresOn[color])
          sleep(0.75)
          self.cajasColores[color].configure(image = self.coloresOff[color])
          if(segundaPausa):
                    sleep(0.75)
```

Este método cambia la imagen de uno de los colores durante un tiempo para simular que se ha encendido y apagado. Recibe dos parámetros:

- **color**: el índice en la lista de colores del que se ha de mostrar.
- **segundaPausa** (opcional): si, después de *apagar* el color se ha de hacer una pausa o no. En el juego, se hace una pausa después de *apagar* cuando se esta mostrando la secuencia al jugador y no cuando el jugador la esta repitiendo.

Para cambiar la imagen, accede a la etiqueta que contiene la imagen a cambiar y, usando el método **configure**, actualiza el objeto.

comprobarBotonColor

```
##
# comprobarBotonColor
# al pulsar alguno de los botones de colores,
# comprueba si la pulsación ha sido correcta,
# según la posición de la secuencia a comprobar
# @pin: el pin del botón que se ha pulsado
##
def comprobarBotonColor(self, pin):
          # se comprueba que los botones no esten bloqueados
          # y que se haya recibido un valor de pin a True
          # (se comprueba el pin para evitar rebotes )
          if(self.botonesBloqueados == False and GPIO.input(pin) == True):
                    # bloquea los botones y, a partir del indice del botón pulsado,
'enciende' el color
                    self.botonesBloqueados = True
                    indiceBoton = self.botonesColores.index(pin)
                    self.mostrarColor(indiceBoton, False)
                    # se comprueba si el pin del botón pulsado es igual
                    # al guardado en la posición a comprobar de la secuencia
                    if(self.secuencia[self.pasoComprobar] == indiceBoton):
                              # el jugador ha acertado, se incrementa el paso a comprobar
                              # si se ha llegado al final de la secuencia, se pasa a
mostrar la siguiente,
                              # si no, se desbloquean los botones y se espera a la
siguiente pulsación
                              self.pasoComprobar += 1
                              if(self.pasoComprobar == len(self.secuencia)):
                                        self.puntos += 1
                                        self.actualizarPuntuaciones()
                                        self.mostrarMensaje(1, "intento.ok")
                                        sleep(1)
                                        self.mostrarMensaje(1, None)
                                        self.lanzarSecuencia()
                              else:
                                        self.botonesBloqueados = False
                    else:
                              # el jugador ha fallado, se muestra el mensaje
                              # de find de juego y el mensaje de bienvenida
                              self.mostrarMensaje(1, "intento.ko")
                              sleep(1)
                              self.mostrarMensaje(0, "juego.terminado")
                              self.mostrarMensaje(1, None)
                              sleep(1)
                              self.mostrarMensaje(0, "bienvenida.1")
                              self.mostrarMensaje(1, "bienvenida.2")
                              self.partidaIniciada = False
```

Este método se llama cada vez que se produce una pulsación en alguno de los botones del colores. El método recibe como parámetro el pin del botón que se ha pulsado.

Antes de hacer nada, el método comprueba que los botones no estén bloqueados y que el pin que se ha pulsado esta en alto. La comprobación del estado del pin se hace para evitar rebotes, en los que el estado del pin esta en bajo.

En caso de que se cumpla la condición, se bloquean los botones y se busca el índice en la lista de botones del pin que se ha recibido como parámetro. El índice servirá para cotejarlo con el índice de imagen guardado en la secuencia a repetir.

Se comprueba si el valor guardado en el paso a comprobar de la secuencia es igual al pulsado por el jugador, y, en caso afirmativo, se incrementa el puntero al paso a comprobar y se comprueba si se ha llegado al final de la secuencia.

Si se ha llegado al final de la secuencia, se incrementa la puntuación y se comprueba si se ha superado la puntuación máxima. Se actualizan estos valores en pantalla, se informa al jugador del acierto y se lanza la siguiente secuencia. Si no se ha llegado al final de la secuencia, se desbloquean los botones para que el jugador siga pulsando hasta acabar la secuencia, o fallar.

En caso de fallo, se informa al jugador por pantalla y se indica que la partida a finalizado, modificando la propiedad de la clase que lo controla.

actualizarPuntuaciones

```
##
# actualizarPuntuaciones
# comprueba si se ha superado el record, actualizando si es necesario,
# y actualiza los textos en sus etiquetas
##
def actualizarPuntuaciones(self):
          if(self.puntos > self.record):
                    self.record = self.puntos
                    self.cajaPuntos.configure(text = self.puntos)
                    self.cajaRecord.configure(text = self.record)
```

Este método simplemente comprueba si se ha superado la puntuación máxima, para actualizarla, y, también actualizar los valores en sus respectivas etiquetas, en la rejilla de la ventana.

La función main

Esta función se ejecutará al iniciar el script y se encargará de instanciar un objeto de la clase **Simon** para que se inicie el juego.

```
##
# main
# función a ejecutar cuando se inicia el script
##
def main():
          # botón inicio juego -> 16
          # botones de colores:
          # 12 -> verde
          # 15 -> amarillo
          # 13 -> rojo
          # 11 -> azul
          # se arranca una instancia de la clase 'Simon'
          Simon(16, 12, 15, 13, 11)
          return 0
          if __name__ == '__main__':
                    main()
```

Se comprueba si el ámbito del módulo ejecutado es __**main**__, y, si es así, se llama a la función **main**.